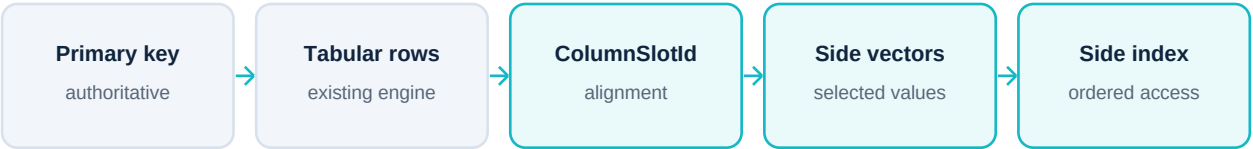


# Tabular + columnar side indexes

A complete expert-review guide to WorkTable's middle storage flavor: authoritative tabular rows plus derived columnar side structures.

|                   |                       |                   |
|-------------------|-----------------------|-------------------|
| SIDECAR ANALYTICS | PRIMARY KEY PRESERVED | FORMAT COMPATIBLE |
|-------------------|-----------------------|-------------------|



|                                           |                                                               |                                                                    |
|-------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------|
| <b>Additive</b><br>Opt in field by field. | <b>Compact</b><br>Choose 8, 16, 32, or 64-bit slot positions. | <b>Safe</b><br>Every result retains the authoritative primary key. |
|-------------------------------------------|---------------------------------------------------------------|--------------------------------------------------------------------|

Review focus: the three-flavor boundary, primary-key authority, bounded-slot behavior, mutation correctness, derived persistence, and honest HANA comparison.

**v3 review boundary: this guide describes the implemented tabular + columnar side-index flavor, and labels the separate full-columnar roadmap explicitly.**

WorkTable can maintain selected fields in derived, chunked **side indexes** while its ordinary row store and primary key remain authoritative. Point-oriented application code keeps the existing WorkTable model, while analytical code gains cheaper columnar-flavored scans and projections over the duplicated selected values.

The first implementation is intentionally format-compatible and uncompressed. It establishes the DSL, identity, mutation, validation, and recovery boundaries before adding sealed chunks, compression, a vector execution engine, or a new disk format.

## The three WorkTable storage flavors

| Flavor                          | Authoritative representation                          | What it provides                                                                                                              | Status                      |
|---------------------------------|-------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------|-----------------------------|
| Tabular                         | Existing WorkTable rows                               | Point lookup, mutation, ordinary indexes and persistence                                                                      | Already well covered        |
| Tabular + columnar side indexes | Existing WorkTable rows, plus derived side structures | Cheap columnar-flavored field scans, projections and ordered lookup at the cost of duplicating selected values/index metadata | <b>This proposal and PR</b> |
| Columnar                        | A true vector/column representation                   | Vector batches, encoded or compressed segments, columnar-native execution and persistence                                     | Not covered                 |

The middle flavor is not a full column store. Calling it one would erase the most useful property of the proposal: it adds a bounded, opt-in analytical acceleration structure without replacing the tabular engine. It is analogous to adding an index, except that a useful side index must duplicate the selected field values as well as ordering metadata. If it stored only keys and slot positions, every projection would still perform random gathers from tabular rows and lose most of its columnar flavor.

# Complete schema

```
use worktable::prelude::*;
use worktable::worktable;

// WorkTable's current column parser accepts a single type identifier.
type DiagnosticBlob = Vec<u8>;

worktable!(
  name: HistoricalCpu,
  persist: true,

  columns: {
    id: u128 primary_key,

    // Bare `columnar` uses the table defaults.
    host_id: u64 columnar,
    captured_at_ns: u64 columnar,
    cpu_percent: f32 columnar,

    // A non-indexed columnar field. It has contiguous field storage and
    // can be scanned or projected even though no index clusters by it.
    status: String columnar(chunk_rows(16_384)),

    // An ordinary row-only field.
    diagnostic_blob: DiagnosticBlob,
  },

  columnar_indexes: {
    host_time: {
      cluster_by: [host_id, captured_at_ns],
    },
  },

  config: {
    columnar_slot_id: ColumnSlotId32,
    columnar_chunk_rows: 65_536,
  },
);
```

Both columnar `config` entries shown above are defaults and may be omitted.

There is deliberately no table-level `layout: columnar`: this proposal does not select the third flavor. A field opts into a derived side index. There is also deliberately no `columns: [...]` list inside a columnar index: fields named by `cluster_by` are its key, and projected values come from base column stores.

## Three independent choices

### 1. Which fields have column storage?

Add `columnar` to a non-primary-key field:

```
cpu_percent: f32 columnar,
status: String columnar(chunk_rows(16_384)),
latency_ns: u64 optional columnar(compression(none)),
```

This duplicates that field in a chunked derived side index. It does not create an ordered search index and it does not change row persistence. A field may be columnar without appearing in any `columnar_indexes` entry; `status` in the complete example is one such field.

`columnar` occurs after `optional` and before `using`. The table default is 65,536 rows per chunk. A per-field `chunk_rows(N)` override must be a power-of-two multiple or divisor of the table default, which keeps cross-column chunk boundaries nestable.

Mutable chunks are currently unencoded. Omitted compression and `compression(none)` mean the same thing. `auto`, `delta`, `rle`, and `dictionary` are reserved and fail macro expansion instead of silently behaving like `none`.

## 2. Which access paths are ordered?

```
columnar_indexes: {
  host_time: {
    cluster_by: [host_id, captured_at_ns],
  },
},
```

`cluster_by` orders the index metadata, not the physical base columns. In the current implementation, the access path is a `BTreeMap` from the composite key to a set of column slot IDs. The base columns remain in canonical slot order.

Every `cluster_by` field must itself declare `columnar`. `include` is reserved for a future genuine covering projection and is rejected today.

## 3. How wide is the compact slot position?

The table-wide setting is:

```
config: {
  columnar_slot_id: ColumnSlotId16,
},
```

Available types and theoretical live-slot capacities are:

| Type                        | Positions                  | Typical reason to choose it                   |
|-----------------------------|----------------------------|-----------------------------------------------|
| <code>ColumnSlotId8</code>  | 256                        | tests or a strictly bounded tiny table        |
| <code>ColumnSlotId16</code> | 65,536                     | embedded or hard-bounded live window          |
| <code>ColumnSlotId32</code> | 4,294,967,296              | default; broad capacity with compact metadata |
| <code>ColumnSlotId64</code> | 18,446,744,073,709,551,616 | explicit very-large logical range             |

These are capacity bounds, not promises that the process can allocate that many rows. Address space, memory, and other table structures impose practical limits first-especially for 64-bit slots.

Choosing a width that covers the maximum number of simultaneously live columnar rows is the schema author's responsibility. WorkTable does not silently widen, truncate, wrap, evict another row, or reinterpret the setting. A write beyond the selected range returns:

```
WorkTableError::ColumnSlotIdExhausted(bits)
```

The failed insert is rolled back and existing rows remain valid. Operators can monitor:

```
table.columnar_slots_in_use();
table.columnar_slots_high_water();
```

## A slot is not an identity or sort key

The primary key remains load-bearing. A `ColumnSlotId` is only a compact position shared by the derived field chunks and columnar indexes. It is not:

- a replacement primary key;
- the row's rank in `cluster_by` order;
- a stable external identifier;

- a durable identifier across restart; or
- a public tuple value applications should store.

Generated results carry an opaque reference:

```
pub struct ColumnarRowRef<PrimaryKey, SlotId> { /* private */ }

impl<PrimaryKey, SlotId> ColumnarRowRef<PrimaryKey, SlotId> {
    pub fn primary_key(&self) -> &PrimaryKey;
}
```

`ColumnarRowRef` deliberately does not implement serialization. Durable application references must store the primary key.

## Delete/reinsert and ABA safety

A bounded slot allocator must reuse positions. Primary key plus slot alone is insufficient: delete and reinsert of the same primary key into the same slot would make an old reference appear valid.

WorkTable therefore validates four pieces of state:

```
primary key + slot position + u64 slot generation + table incarnation
```

The generation is separate from the configured slot width, so choosing `ColumnSlotId8` still gives 256 live slots rather than carving generation bits out of those eight bits. A delete increments the slot's generation before reuse. Generation never wraps: if its `u64` counter is ever exhausted, that slot is permanently retired. A process-local table incarnation invalidates references created by another table instance or before a persisted table is reopened.

This is stronger for retained owned references than relying only on an epoch grace period: an epoch protects active readers, but it cannot protect a reference an application stores after the read guard ends.

## Current side-index physical model

For each generated table with at least one columnar field, WorkTable keeps the tabular row and adds:

```
authoritative primary key -> authoritative WorkTable row/link
                          -> (ColumnSlotId, generation) directory
                          -> chunked base field replicas
                          -> zero or more clustered BTreeMap access paths
```

Each duplicated side field currently uses `Vec<Vec<Option<T>>>`. The outer vector holds chunks and the inner vector is indexed by the slot offset. A separate primary-key column supports reference validation. This first layout is deliberately generic:

- fixed-width values are not yet exposed through SIMD/vector kernels;
- optional fields currently store the Rust `Option<T>` representation rather than a validity bitmap;
- `String` values are owned values rather than offsets into a byte arena; and
- chunks are mutable and uncompressed.

These are implementation boundaries, not compression or vectorization claims.

## Generated API in this implementation

The current generated methods return owned collections:

```
// Exact lookup requires the complete composite clustered key.
let refs = table.columnar_select_host_time(host_id, captured_at_ns)?;

// Gather a selected field through validated opaque references.
let cpu = table.columnar_project_cpu_percent(&refs)?;

// A direct scan needs no columnar index.
let statuses = table.columnar_scan_status()?;

// Scan in the clustered index's key order.
let ordered_refs = table.columnar_scan_host_time()?;
```

The owned **Vec** results keep locks out of the public return type and may be retained safely. They also materialize the result, so this API is not the final high-volume execution surface.

Not yet implemented:

- prefix equality and range predicates on a composite **cluster\_by** key;
- namespaced predicate/projection builders;
- zero-copy or callback-based **scan\_batches**;
- a single multi-field projection API; and
- row-gather fallback for row-only fields.

Those remain the next query-API slice. Documentation and benchmarks must not present them as shipping behavior.

## Mutation and consistency behavior

Insert, ordinary update, delete, and reinsert maintain the derived directory, field chunks, and clustered metadata before the mutation call completes. A same-primary-key update retains its slot and generation. Vacuum may move the authoritative physical link without changing the columnar slot.

The complete derived state is protected by one table-local read/write lock. Consequences:

- an individual scan or projection observes one coherent columnar snapshot;
- all columnar fields changed by one maintenance operation publish together;
- concurrent columnar writers are table-serialized;
- ordinary row-only selects do not take the columnar lock; and
- two separate API calls are two snapshots, not a transactionally pinned multi-call view.

Some archived in-place mutation paths cannot apply a typed column delta yet. They mark the derived replica dirty. The next columnar read rebuilds it from authoritative rows. Applications can see and schedule that cost explicitly:

```
if table.columnar_is_dirty() {
    table.rebuild_columnar()?;
}
```

The current rebuild is whole-table and holds the columnar writer lock. Per-chunk dirty tracking is planned; it is not implemented in this release.

## Persistence and recovery

For **persist: true**, ordinary WorkTable rows and indexes retain their existing formats. The columnar side index is marked derived, omitted from the persisted index structure, and rebuilt from authoritative rows after load. Therefore this change introduces no new on-disk columnar format.

`ColumnSlotId` assignments are not promised to survive restart. The process/table incarnation in `ColumnarRowRef` prevents an old in-memory reference from being accepted by a reopened instance. Use primary keys for durable identity.

Native columnar checkpoints are a later design choice. They should be added only if benchmarked restart time or row-store gather cost justifies another durable format and recovery protocol.

## Relationship to SAP HANA's unified table

Sikka et al.'s SAP HANA paper is the most relevant architectural contrast because it explains how a true column-oriented system can serve transactional and analytical work on one logical table. HANA primarily informs WorkTable's possible third flavor, while this PR deliberately implements the cheaper middle flavor. The similarity is the use of different physical representations for different access patterns; the authority and execution models differ.

| Dimension            | SAP HANA (SIGMOD 2012)                                                    | WorkTable side indexes / PR implementation                                           |
|----------------------|---------------------------------------------------------------------------|--------------------------------------------------------------------------------------|
| Logical goal         | OLTP and OLAP through one unified table interface                         | Preserve the tabular engine and add opt-in columnar-flavored side indexes            |
| Write path           | Uncompressed row-oriented L1 delta                                        | Existing authoritative WorkTable row storage                                         |
| Intermediate form    | Dictionary-encoded, unsorted column L2 delta                              | Mutable, uncompressed side-index vectors                                             |
| Read-optimized form  | Compressed main store with sorted dictionaries and bit-packed values      | Not implemented yet                                                                  |
| Record position      | RowId created on entry; positional alignment across columns               | Primary key is authoritative; opaque slot aligns derived columns                     |
| Reorganization       | Asynchronous L1→L2 and snapshot-safe L2→main merges                       | Synchronous maintenance; whole-table rebuild only for dirty fallback paths           |
| Readers during merge | Old/new versions retained until transactions using the old version finish | One table-local columnar <code>RwLock</code> ; no versioned columnar merge yet       |
| Point access         | Inverted indexes across delta and main structures                         | Generated <code>BTreeMap</code> clustered metadata plus the normal WorkTable indexes |
| Execution            | Row/column iterators and vectorized block-at-a-time operators             | Owning <code>Vec</code> scans/projections in this first API                          |
| Durability           | REDO for incoming changes plus savepoints for column structures           | Existing WorkTable persistence is authoritative; columnar state rebuilds after load  |

The closest honest analogy is: **WorkTable's tabular engine plays a role similar to HANA's write-optimized L1, while this PR adds optional uncompressed side indexes. It does not implement HANA's L2/main column-store lifecycle and does not claim the third, fully columnar flavor.**

The HANA result points to a defensible next architecture for WorkTable:

- keep a small mutable delta that accepts foreground mutations cheaply;
- seal cold column chunks into immutable snapshots;

- build dictionaries, bit packing, delta/RLE, and zone metadata off the foreground path;
- publish a new manifest atomically while readers finish on the previous snapshot;
- reclaim old snapshots only after the read grace period; and
- checkpoint sealed chunks separately from the authoritative row representation only when the recovery and performance measurements justify it.

That architecture belongs to a future full-columnar effort and would also remove the current whole-table dirty-rebuild cliff. It is a roadmap informed by HANA's record-lifecycle design, not a performance claim for this PR or a requirement for the side-index flavor.

Reference: Vishal Sikka, Franz Färber, Wolfgang Lehner, Sang Kyun Cha, Thomas Peh, and Christof Bornhövd. "Efficient Transaction Processing in SAP HANA Database: The End of a Column Store Myth." SIGMOD 2012, pp. 731-741. DOI: 10.1145/2213836.2213946.

## Compile-time validation

The macro rejects:

- `columnar` on a primary-key field;
- an unknown or non-columnar field in `cluster_by`;
- empty or duplicate `cluster_by` entries;
- a columnar field/index generated-method name collision;
- the removed `columns`: index property;
- the reserved `include`: property;
- unsupported compression policies;
- zero or non-nesting chunk sizes;
- duplicate table config entries;
- unknown or out-of-order column attributes; and
- `columnar_indexes` in `worktable_version!`.

## Benchmark contract before performance claims

For each supported primary-index backend (`WorkTablesIndex`, `congee-wt`, and `arctic-wt` where the `Using` and persistence rules permit it), measure:

- row select throughput and p50/p95/p99 with no columnar declarations, fields only, and fields plus clustered metadata;
- insert, same-key update, indexed-key update, delete, and fixed-window delete/reinsert churn;
- full field scan, exact clustered lookup, and projection gather;
- single-thread and 1→core-count concurrent readers/writers;
- memory amplification, allocation count, code size, and slot-directory overhead by width;
- first-read dirty rebuild versus application-scheduled rebuild;
- persisted reload and rebuild time; and
- correctness counters alongside throughput, especially under slot reuse and concurrent mutation.

Until those measurements exist, the safe statement is that the feature adds correctness-tested columnar side indexes-not that it is a full column store, faster for every workload, or ready for latency-sensitive HFT deployment.

## Staged roadmap

- **Query surface:** namespaces predicate builders, prefix/range selection, combined projection, and bounded `scan_batches`.
- **Incremental maintenance:** per-chunk dirtiness and typed in-place deltas.
- **Column encodings:** validity bitmaps, fixed-width vector kernels, and offset/byte storage for variable-width fields.
- **Separate full-columnar design:** mutable delta, sealed immutable chunks, background merge, versioned publication/reclamation, and vector execution. This is a different flavor, not a silent expansion of the side-index feature.
- **Compression:** type-checked dictionary, delta, RLE, bit packing, and an evidence-based `auto`.
- **Optional native persistence:** manifests, checksums, recovery watermarks, and crash tests.
- **Optimizer:** choose row lookup, ordinary index, clustered side-index lookup, or base-field scan from measured costs.

This ordering keeps correctness and compatibility ahead of compression claims while leaving the DSL stable for the later physical evolution.

## Reviewer decision points

- Is "tabular + columnar side indexes" the right permanent name for this middle flavor?
- Is full-width `ColumnSlotId8|16|32|64` plus a separate `u64` generation preferable to hiding a smaller slot/generation bit split inside the configured width?
- Is the owned, fully materialized phase-one API acceptable if `scan_batches` is the next query slice and no vector-execution claim is made now?
- Is rebuild-on-load the correct compatibility choice until native side-index checkpoints show a measured recovery benefit?
- Should the future full-columnar flavor receive distinct DSL rather than changing the meaning of today's field-level `columnar` attribute?